Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Inputs, outputs, and the return contract

Today
Welcome to your next lesson: Inputs, outputs, and the return contract.

I'm getting your personalized lesson ready, focusing on what you've learned, what's ahead, and how to help things stick.

Step by step, you’re moving forward.

Let’s jump in with something concrete, Ron 🔧

Imagine this little Python script:

Python

```
raw = input("Enter temperature in F: ")
fahrenheit = float(raw)
celsius = (fahrenheit - 32) * 5 / 9
rounded = round(celsius, 1)
print("Celsius:", rounded)
```

Right now it’s just a straight script.

Step 1 today is: find the input → output of this chunk.

👉 In your own words:

What would you say is the main input here, and what is the final output this code produces? (Just a quick short answer is fine.)

The main input s the Farenheight Temperature value and the output is the Celsius equivelant of that Temperature

Exactly right 🎯

Input: the Fahrenheit temperature value (coming from input(...), then turned into float)
Output: the Celsius equivalent of that temperature (the rounded value that gets printed)

So this whole script is basically:

“Take a Fahrenheit number in → give me back a Celsius number out.”

Next step: we’re going to box that logic into a function.

First, staying high level:

If we turn this into a function, which would you prefer as the function input?

A: The raw text from input()
B: A numeric Fahrenheit value (like 72.0)

Just tell me “A” or “B” and we’ll build from there.

The temperatore in faren

Nice, that’s the one I’d pick too 👍

So we’ll think of our function like this:

Input: a Fahrenheit number

Output: a Celsius number

In code terms, the “box” will look like:

Python

```
celsius = convert_temp(72.0)
```

Where:

72.0 goes into the function
celsius gets the result that comes out
Before we actually write it as Python, I want to be sure this picture is clear:

Fahrenheit number → [convert_temp] → Celsius number

Does that flow make sense to you as a clean input → output box?

(Yes / No is fine, and if “no”, tell me what feels fuzzy.)

Yes, that makes sense

Great ☑

Now let’s actually turn that script into a function step by step.

Here’s the original again:

Python

```
raw = input("Enter temperature in F: ")
fahrenheit = float(raw)
celsius = (fahrenheit - 32) * 5 / 9
rounded = round(celsius, 1)
print("Celsius:", rounded)
```

We want a function shaped like:

Python

```
def convert_temp(fahrenheit):
    # use fahrenheit
    # end with: return <celsius_value>
```

Key change: instead of reading with input() inside the function, we pass in fahrenheit as a parameter, and instead of print, we'll later make it return.

Let's do this in the editor together:

Open the code editor.

Copy the four "middle" lines (everything except the input and the print) into a function:

```
def convert_temp(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
    # we'll decide what to do here next
```

Below the function, add a test call, like:

```
result = convert_temp(72.0)
print(result)
```

Don't worry yet about return vs print inside the function—we'll fix that next.

Go ahead and build this in the editor, hit Run to see what happens, then Submit code so I can look at it.

it outputs None currently

Perfect observation 👌 — that None is a big clue.

Right now your function probably looks like:

Python

```
def convert_temp(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
    # no return here
```

In Python:

If a function doesn't hit a return with a value, it automatically returns None.

So when you do:

Python

```
result = convert_temp(72.0)
print(result)
```

result is None, so it prints None.

Next tiny step:

Inside your function, on the last line, add:

Python

```
return rounded
```

So the full function is:

Python

```
def convert_temp(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
```

```
    return rounded
```

Then run again with:

Python

```
result = convert_temp(72.0)
print(result)
```

Try that in the editor, Run it, and tell me what it prints now.

22.2

Nice, that’s exactly what we want 🎯

Now your function has a clear input → output contract:

Input: fahrenheit (a number, like 72.0)
Output: the rounded Celsius value (like 22.2), via return

So we’ve hit outcome #1: you extracted a clean function from script code.

Next concept: return vs print inside a function.

Imagine two versions of convert_temp:

Version A:
Python

```
def convert_temp(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
    print(rounded)
```

Version B:
Python

```
def convert_temp(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
    return rounded
```

Both show 22.2 on the screen if you print the result somehow, but they behave very differently.

Key idea:

print just sends text to the screen.

return sends a value back to the caller, so your code can keep using it.

Quick check: does that difference feel clear right now, or a bit fuzzy?

You can answer with something like: “clear” or “still fuzzy”.

It's clear now

What do you think? Pick the option that best fits, then we’ll look at how to actually use it in code 👇

You have these two functions:

```
def convert_temp_print(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
```

```
    print(rounded)


def convert_temp_return(fahrenheit):
    celsius = (fahrenheit - 32) * 5 / 9
    rounded = round(celsius, 1)
    return rounded
```

You now want to double the Celsius value and then print it. For example, if 72.0°F → 22.2°C, you want to print 44.4.

Which function is easier to use for this new goal inside another function or calculation?

A.

convert_temp_print is easier, because it already prints the value.

B.

convert_temp_return is easier, because it gives back a value I can reuse.

C.

Both are equally easy to reuse in other calculations.

🎉 Nailed it, Ron.

You picked the right one:

convert_temp_return is easier, because it gives back a value I can reuse.

Because it returns the Celsius number, you can do things like:

Python

```
c = convert_temp_return(72.0)
double_c = c * 2
print(double_c)
```

With the print version, the value disappears after printing — you can't grab it and multiply it. That's why return keeps your functions composable and flexible.

Let's lock this in with a tiny real-world refactor next:

we'll turn a repeated sales tax calculation into a function like apply_tax(price).

Ready for that next step? (Yes/No is fine.)

Yes, I'm ready

Awesome 😄 Let's do a small, concrete one.

Say you have this script that adds sales tax (8%) to two prices:

Python

```
price1 = 50.0
tax1 = price1 * 0.08
total1 = price1 + tax1
print("Total 1:", total1)

price2 = 120.0
tax2 = price2 * 0.08
total2 = price2 + tax2
```

print("Total 2:", total2)
You can see the same pattern repeated:

price → compute tax → add to price → get total

We want to extract that into a function with a clear contract:

Input: price

Output: total_with_tax

So we’re aiming for something like:

Python

total1 = apply_tax(50.0)
total2 = apply_tax(120.0)
Your turn in the code editor 🧠💻

Open the editor.

Create a function:

Name: apply_tax
Parameter: price
Inside: calculate total with 8% tax.
End with: return <total_with_tax_value>
Below it, call your function twice (for 50.0 and 120.0) and print the totals.

Run it, then Submit code so I can check it.

Python

```
def apply_tax(price):
    tax = price * 8.00
    total = price + tax
    return(total)
print(f"50.0  with 8% Tax is: {apply_tax(50):.2f}")
print(f"120.0  with 8% Tax is: {apply_tax(120):.2f}")
```

Nice work getting that function in place, Ron 👍

You’ve got the right structure:

Python

```
def apply_tax(price):
    tax = price * 8.00
    total = price + tax
    return total
```

and you’re using the returned value correctly:

Python

```
print(f"50.0  with 8% Tax is: {apply_tax(50):.2f}")
print(f"120.0  with 8% Tax is: {apply_tax(120):.2f}")
```

So from a functions + return point of view, this is correct:

clear input: price
clear output: total returned
no print inside the function 👍

The only small issue is the math for 8%:

price * 8.00 means “800% of price”
for 8%, you want price * 0.08

So inside your function, change:

Python

```
tax = price * 8.00
```

to:

Python

```
tax = price * 0.08
```

Run again and check that:

50.0 → 54.00
120.0 → 129.60
After you fix that and run it, tell me:

Do you feel solid now on why we return total instead of printing it inside apply_tax? (Yes / Not really)

no dought not awake yet i gues io knoew that

Love the honesty 😄 and your code shows you do get it.

Let’s lock the idea in super clearly, step by step.

Think of two versions of apply_tax:

Python

```
def apply_tax_print(price):
    tax = price * 0.08
    total = price + tax
    print(total)   # only prints, no return

def apply_tax_return(price):
    tax = price * 0.08
    total = price + tax
    return total   # no print here
```

Visually:

print version:
price → function → 💬 shows total on screen → nothing comes back

return version:
price → function → ☑ sends total back to the caller → you can still print, store, compare, add, etc.

That’s why your code like this works:

Python

```
print(apply_tax(50))
```

Because apply_tax returns a number, and print uses that number.

Quick check in your own words (super short):

Why is the return version more flexible than the print version?

Python

```
def apply_tax(price):
    tax = price * .08
    total = price + tax
    return(total)
print(f"50.0  with 8% Tax is: {apply_tax(50):.2f}")
print(f"120.0  with 8% Tax is: {apply_tax(120):.2f}")
```

This looks great now, Ron 🎯

```
def apply_tax(price):
    tax = price * .08
    total = price + tax
    return(total)

print(f"50.0  with 8% Tax is: {apply_tax(50):.2f}")
print(f"120.0  with 8% Tax is: {apply_tax(120):.2f}")
```

Clear input → output: price goes in, total comes back out.
You use return inside the function, and print outside where you decide how to display it.
The math is correct: 50.0 → 54.00, 120.0 → 129.60 ☑
You've hit both goals for this lesson:

You can spot a chunk of script and turn it into a function with a clean input/output boundary.
You understand why return makes functions reusable, while print is just for showing things on screen.
Quick recap to keep in mind:

Use return for data you want to reuse (store, combine, pass to other functions).
Use print only when you want to show something to the user.
You're building good habits here. Functions with clear contracts will make your future code much easier to read and grow 🚀

When you're ready, you can continue to the next lesson from the Learn tab.